SONIC AI

Complete Assignment Build Guide

Collections Agent + Bank Reconciliation Platform

Step-by-step implementation handbook for Antigravity

Frontend • FastAPI Backend • MongoDB • Email/LLM Workflow • CSV Reconciliation • Testing • Submission

Prepared from the Sonic AI internship assignment and the implementation plan discussed in this chat.

Important: This is a practical build guide, not a claim that the application has already been built or tested. Run every stage, inspect the generated code, and test it with your own demo mailboxes and database.

1. Project ka objective

Tumhe AcmeCorp ke liye ek collections aur bank-reconciliation application banana hai. System ko invoices track karne, customers ko payment reminders bhejne, customer email replies samajhne, aur bank CSV transactions ko invoices se match karne mein help karni hai. Sensitive decisions human approval ke bina nahi hone chahiye.

Core product modules

• **Organization:** company/bank details aur collection policies dikhana/edit karna.

• **Customers:** customer list aur email address edit karna.

• **Invoices:** invoice status, due date, total, paid amount aur outstanding balance.

• **Collections:** overdue invoices ke liye reminder emails bhejna.

• **Conversations:** customer replies ko receive, classify aur track karna.

• **Needs Attention:** disputes, extensions, legal threats, unknown senders aur sensitive requests ko human review mein bhejna.

• **Reconciliation:** bank CSV upload, invoice matching suggestions, confidence score aur human approval.

• **Payment history:** approved receipts aur invoice balance changes ka audit trail.

2. Assignment ke important rules

Invoice aur overdue rules

• Invoice ka customer relation `customer_id` se hota hai.

• Overdue status database mein alag se store nahi hota. Calculate karo: invoice status `sent` ho, `balance_due > 0` ho, aur `due_date` aaj se pehle ho.

• Draft aur already-paid invoices ke liye reminder nahi bhejna.

• Collections answer mein sirf usi customer ke sent aur unpaid invoices ka context use karna.

• AI ke generated invoice number, dates aur amounts ko send karne se pehle database ke against validate karna.

Organization policy rules

• Indian customers invoice subtotal par 10% TDS deduct kar sakte hain; unhe TDS certificate share karna hoga.

• International customers bank charges cover karenge.

• Later payment date ya installment plan sirf Accounts Team approve kar sakti hai.

• Disputes ko 3 working days ke andar review karna hai.

• Working hours: Monday–Friday, 10:00–18:00 IST.

TDS example: subtotal ■300,000 + GST ■54,000 = invoice total ■354,000. TDS 10% of subtotal = ■30,000. Expected bank receipt ■324,000. Partial-payment/TDS closure ka final decision human approval ke baad hi apply karo.

Customer email safety

• Sample customer emails `example.com` par hain; ye real delivery inboxes nahi hain. Testing ke liye apne test inboxes replace karo.

• Organization ka `accounts_email` bhi test inbox se replace karo.

• Development ke liye do mailboxes use karo: organization mailbox (SMTP + IMAP) aur customer/test mailbox.

3. Recommended technology stack

Layer	Recommended tools	Role
Frontend	React + TypeScript + Vite	Dashboard and user interface
UI	Tailwind CSS + shadcn/ui + Lucide + Recharts	Modern B2B fintech interface
Backend	Python + FastAPI + Pydantic	API and business logic
Database	MongoDB Atlas / local MongoDB + PyMongo	NoSQL storage
LLM	Gemini API	Customer reply classification and draft response
Email	SMTP + IMAP	Send reminders and receive replies
Matching	RapidFuzz + local scoring algorithms	Deterministic bank-to-invoice matching
Testing	Pytest + Vitest	Backend and frontend tests
Scheduling	APScheduler (optional)	Periodic IMAP polling/reminder jobs

Important architecture rule: Frontend sirf backend API ko call karega. Frontend se direct database, SMTP/IMAP, LLM API ya secret keys access nahi honi chahiye. Business rules backend mein rahenge.

4. Project folder structure

```
sonic-ai/
■■■ frontend/
■   ■■■ src/
■       ■■■ components/
■       ■   ■■■ layout/
■       ■   ■■■ dashboard/
■       ■   ■■■ invoices/
■       ■   ■■■ customers/
■       ■   ■■■ collections/
■       ■   ■■■ reconciliation/
■       ■■■ pages/
■       ■   ■■■ Dashboard.tsx
■       ■   ■■■ Organization.tsx
■       ■   ■■■ Customers.tsx
■       ■   ■■■ Invoices.tsx
■       ■   ■■■ Collections.tsx
■       ■   ■■■ Conversations.tsx
■       ■   ■■■ Reconciliation.tsx
■       ■■■ services/
■       ■■■ hooks/
■       ■■■ types/
■       ■■■ App.tsx
■■■ backend/
■   ■■■ app/
■       ■■■ api/routes/
■       ■■■ core/
■       ■■■ database/
■       ■■■ models/
■       ■■■ schemas/
■       ■■■ services/
■       ■   ■■■ gemini_service.py
■       ■   ■■■ email_service.py
■       ■   ■■■ csv_service.py
■       ■■■ agents/
■       ■■■ reconciliation/
■       ■■■ business/
■       ■■■ jobs/
■       ■■■ main.py
■   ■■■ tests/
■■■ data/
■   ■■■ organization.json
■   ■■■ customers.json
■   ■■■ invoices.json
■   ■■■ bank_statement_sample.csv
■   ■■■ data-explanation.md
```

```
■■■ scripts/
■   ■■■ seed_database.py
■■■ docs/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

5. MongoDB collections / data model

Collection	Purpose
organizations	Company profile, bank details, policies and accounts email
customers	Customer identity, contact email and country
invoices	Invoice totals, currency, due date, status and balance_due
conversations	Customer email thread and review state
messages	Inbound/outbound email content and metadata
payments	Approved receipts and invoice allocations
bank_transactions	Imported bank CSV rows
reconciliation_suggestions	Candidate invoice matches, scores and reasons
reminder_logs	Reminder send history and idempotency keys
review_queue	Human review tasks and draft responses
audit_logs	Who approved/rejected/changed what and when

• Money values ke liye Decimal/Decimal128 use karo; floating-point rounding errors avoid karo.

• Negative balance allow mat karo.

• Bank transaction import ko unique reference/transaction key se deduplicate karo.

• Payment approval idempotent ho: same approval dobara click hone par duplicate payment na bane.

• Payment record aur invoice balance update ko atomic transaction/consistent operation mein handle karo.

6. Backend API endpoints

Method + endpoint	Use
GET /api/organization	Organization details
PUT /api/organization	Organization details update
GET /api/customers	Customer list
PUT /api/customers/{id}	Customer email/details edit
GET /api/invoices	Invoice list and filters
GET /api/invoices/overdue	Overdue invoices
POST /api/demo/reset	Original demo data restore; demo conversations/payments clear
POST /api/collections/remind	Reminder send action
POST /api/email/check-inbox	Manual IMAP inbox check
GET /api/conversations	Conversation list
GET /api/reviews	Needs Attention queue

Method + endpoint	Use
POST /api/reviews/{id}/send	Human-edited draft send
POST /api/reconciliation/upload	Upload and parse bank CSV
GET /api/reconciliation	Transactions and match suggestions
POST /api/reconciliation/{id}/approve	Approve suggested match
POST /api/reconciliation/{id}/reject	Reject suggestion
POST /api/reconciliation/{id}/select	Manually select invoice
GET /api/payments	Payment history

7. Collections agent: end-to-end workflow

Step 1 — Find eligible invoices: backend sent + unpaid + past-due invoices calculate kare. Draft/paid invoices skip hon.

Step 2 — Create reminder: invoice facts database se lo; email template mein verified invoice number, due date, currency, balance aur payment instructions insert karo.

Step 3 — Send with SMTP: send status, timestamp, recipient, message ID aur idempotency key log karo.

Step 4 — Receive replies with IMAP: periodic poll aur manual “Check inbox now” dono support karo. `Message-ID`, `In-Reply-To`, `References` aur sender email se correct conversation identify karo.

Step 5 — Identify sender: customer record se sender match karo. Unknown sender ko sensitive company/customer information disclose mat karo; review queue mein bhejo.

Step 6 — Classify and draft: Gemini ko sirf relevant organization policies aur us particular customer ke invoices ka minimum context do. Full customer database ya doosre customers ke invoices kabhi mat bhejo.

Step 7 — Validate: AI response ke invoice number, due date, amount aur balance ko database facts se verify karo. Mismatch par auto-send block karo.

Step 8 — Human review: sensitive cases mein operator draft edit karke manually send kare.

Email classification categories

Category	Required behavior
routine_query	Verified balance/invoice/payment method ka answer draft/auto-answer
payment_promise	Acknowledge and record promise; invoice paid mark mat karo
payment_claim	Acknowledge claim; bank/payment proof verify hone tak paid mark mat karo
extension_request	Human review; due date extend karne ka promise mat karo
dispute	Human review; 3 working days policy ke mutabik review
legal_threat	Urgent human review; legal position/commitment mat do
unknown_sender	No disclosure; human review
out_of_scope	Safe acknowledgement or human review

Golden rule: customer ka kehna ki payment kar diya hai ya payment date promise karna, apne aap invoice ko paid nahi banata.

8. Bank reconciliation engine

CSV processing

• CSV upload endpoint file type, file size, encoding aur required columns validate kare.

• Rows parse karke normalized transaction records banao.

• Debit rows ko invoice receipt matching se exclude karo.

• Duplicate transaction reference/import ko ignore ya flag karo.

• Bank statement directly LLM ko mat bhejo; local matching algorithms first use karo.

Suggested matching signals

Signal	Initial weight (engineering starting point)
Invoice number/reference match	30%
Payer/customer name similarity	30%
Amount similarity	25%
Date proximity	10%
Currency match	5%

Ye weights assignment ka fixed formula nahi hain; implementation ke starting weights hain. Sample data aur test cases ke basis par tune/calibrate karo.

• Name normalization: punctuation, bank prefixes, extra spaces aur legal suffixes (PVT LTD, LTD, GMBH etc.) normalize karo.

• RapidFuzz `token_set_ratio` ya similar fuzzy string score use kar sakte ho.

• Invoice number exact match strongest evidence ho sakta hai.

• Currency mismatch ko hard reject ya severe penalty do.

• Amount match mein exact, partial aur TDS-adjusted cases alag identify karo.

• Date proximity supporting signal hai; date alone par invoice match approve mat karo.

Reconciliation UI and human approval

• Har credit transaction ke liye likely invoice, confidence score aur one-line reason dikhao.

• Agar reasonable candidate nahi hai to “No match” suggest karo.

• Operator approve, reject ya manually invoice select kar sake.

• Payment sirf approval ke baad apply ho.

• Partial payment par human choose kare: “Close invoice (TDS deducted)” ya “Leave balance open”.

• Balance zero hone par invoice paid mark ho aur further chasing stop ho.

• Multi-invoice payment bonus feature ke roop mein support kar sakte ho.

9. Sample bank statement: test cases

Date	Description (short)	Credit	Currency
2026-09-05	KESTREL FOODS / INV 1004	59,000	INR
2026-09-10	ORBIS PKG / PART PAYMENT	200,000	INR
2026-09-21	QUILLON TECH	324,000	INR

Date	Description (short)	Credit	Currency
2026-09-25	NORDLYS RETAIL / RE 1024	9,200	EUR
2026-09-28	HARBOUR ANALYTICS / INV1021	12,500	USD
2026-09-30	BRIGHTLINE LOGISTICS / INV-1002	236,000	INR
2026-10-02	REDWOOD CLINIC	59,000	INR
2026-10-05	OFFICE RENT OCT (debit)	—	INR
2026-10-08	THORNFIELD MEDIA	7,350.50	GBP
2026-10-12	SAFFRON PEAK HOSP / OCT DUES	194,700	INR
2026-10-15	SOFTWARE VENDOR (debit)	—	USD
2026-10-20	R K SHARMA / CLINIC DUES	88,500	INR
2026-10-24	MAPLE RETAIL SERVICES	12,750	INR
2026-10-31	INTEREST PAID	431.22	INR

Assignment dataset mein 14 statement rows hain; debit rows (office rent, software vendor) invoice receipts nahi hain. Interest credit bhi customer invoice payment nahi maana jana chahiye jab tak evidence na ho.

Example: Quillon Tech ka ■324,000 receipt TDS example ke saath consistent ho sakta hai, lekin actual invoice/customer mapping ko provided JSON data se verify karke hi approve karo.

10. Frontend pages and design

Page	What to show
Dashboard	Overdue amount, outstanding amount, recent receipts, review count
Organization	Organization details, bank details and policies
Customers	Customer table, search/filter, editable email
Invoices	Invoice list, status, due date, total, paid and balance
Collections	Eligible overdue invoices and reminder actions
Conversations	Threaded email conversations and AI draft
Needs Attention	Human review tasks with edit/send controls
Reconciliation	CSV upload, transaction rows, suggestions, confidence and actions
Payment History	Approved payments and allocations
Settings	Email connection status, LLM config status and demo reset

Visual direction: polished B2B fintech dashboard, clean whitespace, clear tables, status chips, restrained colors, readable typography. Linear/Stripe/Vercel-style product quality is a useful design reference. Avoid a generic template look.

11. Antigravity mein build karne ka sequence

Ek hi prompt mein poora product generate karne ke bajay stage-by-stage build karo. Har stage ke baad app run karo, errors fix karo aur Git commit banao.

Stage 1 — Foundation

```
You are a senior full-stack engineer. Build the foundation for a production-quality
Sonic AI collections and bank reconciliation app.

Use React + TypeScript + Vite for frontend, Python FastAPI for backend,
MongoDB with PyMongo, and Pydantic schemas. Create separate frontend and
backend folders. Keep all business logic and secrets in backend.

First inspect the provided assignment files in the data folder. Create the
folder structure, backend health endpoint, frontend shell, CORS config,
MongoDB connection module, settings loaded from environment variables,
.env.example, and .gitignore. Add clear README setup instructions.

Do not implement all features yet. Do not invent fields absent from source data.
Run the backend and frontend, fix startup errors, and report files created,
commands to run, and remaining issues. Stop after the foundation.
```

Stage 2 — Seed data and core screens

```
Continue the existing project; do not replace working code. Implement MongoDB
repositories and seed_database.py using the supplied organization.json,
customers.json and invoices.json. Add organization, customer and invoice APIs.
Implement demo reset to restore original seed data and clear demo conversations
and payments. Build Dashboard, Organization, Customers and Invoices pages.
Calculate overdue from sent + balance_due > 0 + due_date before today.
Allow customer email editing. Add validation and backend tests. Run everything
and fix errors before stopping.
```

Stage 3 — Collections and email

```
Implement SMTP outbound reminders and IMAP inbound email polling, including
a manual Check inbox now endpoint. Use environment variables for credentials.
Match inbound messages to customers/conversations using sender and email thread
headers. Use Gemini only from the backend. Give the LLM only organization policy
and the matched customer's own sent/unpaid invoices. Classify replies as routine
query, payment promise, payment claim, extension request, dispute, legal threat,
unknown sender or out of scope. Never mark paid based on a claim or promise.
Route sensitive/unknown cases to Needs Attention. Validate every generated
invoice number, date and amount against MongoDB before sending. Add tests.
```

Stage 4 — Reconciliation

```
Implement CSV upload and parsing for the supplied bank statement format.
Normalize payer names and references. Exclude debit transactions and duplicate
imports. Build deterministic invoice matching using invoice number, payer
similarity, amount, currency and date. Use RapidFuzz where useful. Return
candidate invoice, confidence score and a concise reason, or no match.
Build approve/reject/manual-select actions. Apply payment only after explicit
human approval. Support partial payments and human choice to close as TDS
deduction or leave balance open. Make approvals idempotent and add audit logs.
Do not send raw bank statements to Gemini. Add comprehensive tests.
```

Stage 5 — Frontend integration and polish

```
Connect every frontend page to FastAPI; do not use mock data in final flows.
Build polished responsive B2B fintech UI for Dashboard, Organization, Customers,
Invoices, Collections, Conversations, Needs Attention, Reconciliation and
Payment History. Add loading, empty, error and success states; confirmation
dialogs for payment approval/reset; clear confidence and review indicators.
Ensure frontend never accesses MongoDB, SMTP/IMAP or Gemini directly.
Run both apps and fix all integration issues.
```

Stage 6 — QA and submission

```
Audit the complete application against the assignment. Test draft/paid invoices
are not chased; overdue sent invoices can be reminded; customer replies are
threaded; customer data is isolated; payment promises/claims do not mark paid;
disputes/extensions/legal threats/unknown senders go to review; AI facts are
validated; CSV debit and duplicate rows are excluded; matching suggestions
include reason and confidence; no payment applies before approval; partial/TDS
choices work; duplicate approval does not double count; demo reset restores
original data. Check secrets are absent from git. Update README, create
docs/architecture.md and docs/api.md, and prepare the explanation PDF and demo
checklist. Run all tests and list any remaining limitations honestly.
```

Antigravity rule: pehle generated files read karo, terminal output dekho, errors copy karke fix karwao. AI-generated code ko bina run/test kiye complete mat samajhna.

12. Environment variables

```
APP_ENV=development
APP_SECRET_KEY=replace_with_random_secret
MONGODB_URI=mongodb://localhost:27017
MONGODB_DATABASE=sonic_ai

GEMINI_API_KEY=your_gemini_api_key
GEMINI_MODEL=your_supported_gemini_model

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your_organization_email
SMTP_PASSWORD=your_organization_app_password

IMAP_HOST=imap.gmail.com
IMAP_PORT=993
IMAP_USERNAME=your_organization_email
IMAP_PASSWORD=your_organization_app_password

CUSTOMER_TEST_EMAIL=your_customer_test_email
FRONTEND_URL=http://localhost:5173
```

`.env` ko GitHub par commit mat karna. `.env.example` commit karo, jisme placeholders hon. Gmail use kar rahe ho to 2-Step Verification enable karke App Password generate karo aur IMAP access enable/check karo. Actual credentials kabhi chat ya source code mein paste mat karo.

13. Testing checklist

• Draft invoice ko reminder nahi jana chahiye.

• Paid invoice ko reminder nahi jana chahiye.

• Overdue sent invoice reminder ke liye eligible honi chahiye.

• Routine query ka answer sirf matched customer ke invoice data se ho.

• Payment promise se invoice paid na ho.

• Payment claim se invoice paid na ho; reconciliation/verification chahiye.

• Extension request, dispute aur legal threat Needs Attention mein jaye.

• Unknown sender ko customer/company data disclose na ho.

• AI incorrect invoice number/amount/date generate kare to send block ho.

• CSV debit row invoice matching se exclude ho.

• Duplicate transaction import/approval duplicate payment create na kare.

• Partial payment aur TDS close/open decisions sahi work karein.

• Payment approval ke bina invoice balance update na ho.

• Zero balance par invoice paid ho aur further chasing stop ho.

• Demo reset original organization/customers/invoices restore kare aur demo conversations/payments clear kare.

14. Security and reliability checklist

• Secrets only backend environment variables mein.

• Customer-specific data isolation: ek customer ke email context mein doosre customer ka invoice nahi.

• CSV MIME/type/size/columns validate karo; malformed input safely reject karo.

• LLM output ko untrusted input samjho; structured schema + database validation.

• Email sender identity verify/match karo; unknown sender ko no-disclosure path.

• Reminder, email polling aur payment approval idempotent rakho.

• Important actions ke audit logs rakho.

• Error logs mein passwords, API keys ya sensitive email content leak na ho.

• Payment/invoice balance updates consistent aur concurrency-safe hon.

15. Submission deliverables

Deliverable	Requirement
GitHub repository	Public repository with complete frontend/backend code
README.md	Setup, run commands, database setup, environment variables and configuration
explanation.pdf	2–3 page explanation PDF (assignment requirement)
Demo video	Unlisted video, approximately 2.5–3 minutes
Working demo	Evaluator apne email accounts aur LLM key se app run/test kar sake

Suggested 3-minute demo flow

• 0:00–0:20 — Product overview and dashboard.

• 0:20–0:50 — Organization, customers and invoices; show overdue/outstanding.

• 0:50–1:25 — Send reminder and show customer reply/conversation.

• 1:25–1:55 — Needs Attention review and human-edited draft.

• 1:55–2:40 — Upload bank CSV, show match confidence/reason, approve one receipt.

• 2:40–3:00 — Show updated invoice/payment history and summarize safety controls.

16. Build priority and time estimate

Phase	Estimated effort
Foundation and environment	2–3 hours
Seed data, core APIs and base screens	3–4 hours
Collections, SMTP/IMAP and LLM	5–7 hours
CSV reconciliation and approvals	5–7 hours
Frontend integration and polish	3–4 hours
QA, documentation and submission	4–6 hours

Ye rough planning estimate hai, guarantee nahi. Agar deadline close/past ho, pehle reliable core flows complete karo: invoice/customer dashboard, safe reminder/reply review, CSV suggestions with human approval, README and honest demo.

17. Documentation/research to cover

• Stripe documentation: revenue recognition, contracts/schedules, pricing models, invoicing and usage concepts.

• Sequence documentation: revenue operations / billing and collections concepts relevant to the assignment.

• Use official documentation for current API behavior and cite links in your project documentation if you rely on them.

Note: Is guide mein documentation topics list kiye gaye hain; external documentation ka detailed research/summarization yahan include nahi kiya gaya.

18. Final practical checklist

• Project root folder create karo aur assignment ZIP extract karo.

• Antigravity mein project folder open karo.

• Stage 1 prompt paste karo; files and terminal output inspect karo.

• MongoDB local ya Atlas setup karo; `.env` configure karo.

• Stage 2 se 6 sequentially run karo; har stage ke baad test + Git commit.

• Apne organization/customer test mailboxes se SMTP/IMAP test karo.

• Sample CSV upload karo; suggestions manually verify karo.

• All security and QA checklist items run karo.

• README, explanation.pdf, public GitHub repo aur unlisted demo video finalize karo.

Deadline note: assignment email timestamp aur 48-hour window ke hisaab se deadline likely 7 November 2026 ke aas-paas thi. Agar submission window pass ho chuki ho, phir bhi project complete karke ASAP submit karo aur recruiter ko short, honest update bhejo. Exact deadline email mein confirm karna.